



Seismosense: Earthquake Detection with Arduino

In Partial Fulfillment of the Requirements in
CST11/L (4406)

Submitted by:

Joanne Faith N. Cabarde

John Erol B. Miano

Jan Loren T. Odiong

John Llorie Z. Sarmiento

Submitted to:

Prof. Nickler S. Seraña

December 2025

I. PROJECT OVERVIEW

This project is a compact, microcontroller-based system designed to detect vibrations associated with earthquakes or structural instability. Unlike simple vibration switches, this system uses a 3-axis accelerometer (ADXL335) to measure acceleration in real-time.

It features an Auto-Calibration routine upon startup to determine the "zero" point of the sensor, regardless of the surface angle. When the combined magnitude of vibration on the X, Y, and Z axes exceeds a pre-defined threshold, the system triggers a visual and auditory alarm.

II. PROJECT OBJECTIVE

A. General Objective:

The main objective of this project is to design, construct, and test a low-cost, reliable, and functional seismic detection system capable of identifying and alerting users to significant ground vibrations characteristic of an earthquake.

Overall goal of the project

B. Specific Objectives:

- To continuously sample vibration data on three axes (X, Y, Z).
- To mathematically combine 3-axis data into a single sensitivity value (Total Vibration Magnitude).
- To ignore the static force of gravity and surface tilt, triggering only on sudden movements.
- To provide immediate feedback via LCD text, LED indicators, and a pulsing buzzer.

III. PROJECT CONCEPT

The Seismosense system functions on the principle of vector magnitude analysis to accurately detect seismic activity while filtering out false positives caused by device orientation. Unlike simple tilt switches, this project utilizes a 3-axis analog accelerometer to continuously measure acceleration forces. Upon startup, the microcontroller executes an automated calibration routine, sampling the environment one hundred times to establish a "zero" baseline that accounts for gravity and surface angle. During active monitoring, the system applies the Euclidean

distance formula to the X, Y, and Z axis data in real-time to calculate a single vibration magnitude score. If this calculated magnitude exceeds a specific sensitivity threshold, the system logic triggers a non-blocking alert sequence, ensuring that even brief but intense shocks result in a sustained warning.

IV. EXPECTED OUTPUT

Upon powering the device, the user will initially see a "Calibrating" message on the LCD screen as the system silently establishes its baseline reference. Once the setup is complete, the display transitions to a monitoring interface, showing real-time differential values for the X, Y, and Z axes, which will remain near zero while the device is stationary. When a vibration simulates an earthquake, the system will immediately react by activating the red LED and emitting a pulsing audible alarm via the buzzer. Simultaneously, the LCD screen will clear to display a critical "EARTHQUAKE" warning alongside the specific magnitude of the shock. This alert state will persist for exactly five seconds before the system automatically resets the alarm, clears the display, and returns to the standard monitoring mode.

V. MATERIALS OR TOOLS NEEDED

Hardware/Physical Tools:

- Arduino Nano Board
- ADXL335 Accelerometer
- 16x2 I2C LCD Display
- Buzzer
- LED
- Resistor 330-ohm
- Jumper Wires
- Breadboard

Software/Simulation Tools:

- Arduino IDE (Integrated Development Environment)
- Libraries: Wire.h, LiquidCrystal_I2C.h, math.h

VI. CONCEPTUAL FRAMEWORK

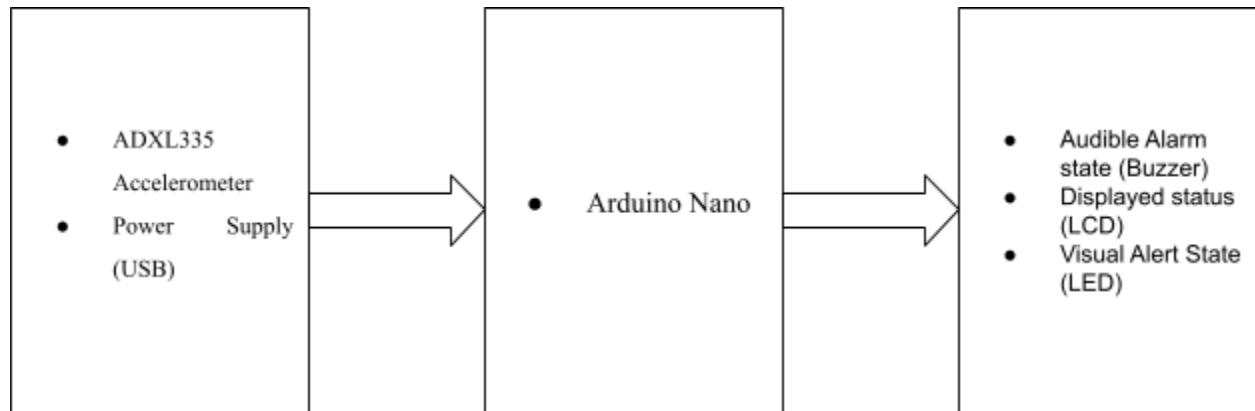


Figure 1. Conceptual Framework

VII. SYSTEM ARCHITECTURE

The system operates as a real-time seismic monitor. It filters out static gravity and reacts instantly to kinetic energy, providing feedback only while the event is occurring.

Main Parts:

Arduino (The Brain): The microcontroller that processes data and executes the logic.

ADXL335 (The Sensor): An accelerometer that continuously measures acceleration forces on three axes (X, Y, and Z).

Buzzer & LED (The Alarm): The active alert system provides audio-visual warnings during the event.

LCD Screen (The Display): Shows real-time difference data and switches to a warning screen during vibrations.

How It Works (Process Flow):

Calibration (Auto-Taring): When powered on, the system executes a setup routine. It takes 100 samples (not 50) from the sensor to calculate an average "Baseline." This effectively tells the Arduino what "stillness" looks like for the device's current position.

Monitoring (Data Acquisition): The Arduino enters a loop where it constantly reads raw data from the X, Y, and Z pins of the sensor.

Processing (Vector Analysis): Instead of just checking one direction, the system calculates the **Magnitude (Total Intensity)** of the vibration. It calculates the difference between the current reading and the baseline for all three axes, then combines them using a mathematical formula (Euclidean distance).

Detection (Trigger Logic): The system compares the calculated Magnitude against a specific sensitivity Threshold of 5. If the vibration intensity exceeds this number, the system identifies it as an earthquake event.

Alert (Warning State): Upon detection, the system triggers the "Alarm State." The Buzzer pulses a 1000Hz tone, the LED flashes, and the LCD clears to display "EARTHQUAKE !!!!" along with the specific magnitude rating. This state is locked for exactly 5 seconds to ensure the warning is seen and heard.

Reset: Once the 5-second timer expires, the system automatically shuts off the buzzer/LED, clears the screen, and returns to continuous Monitoring mode, ready for the next event.

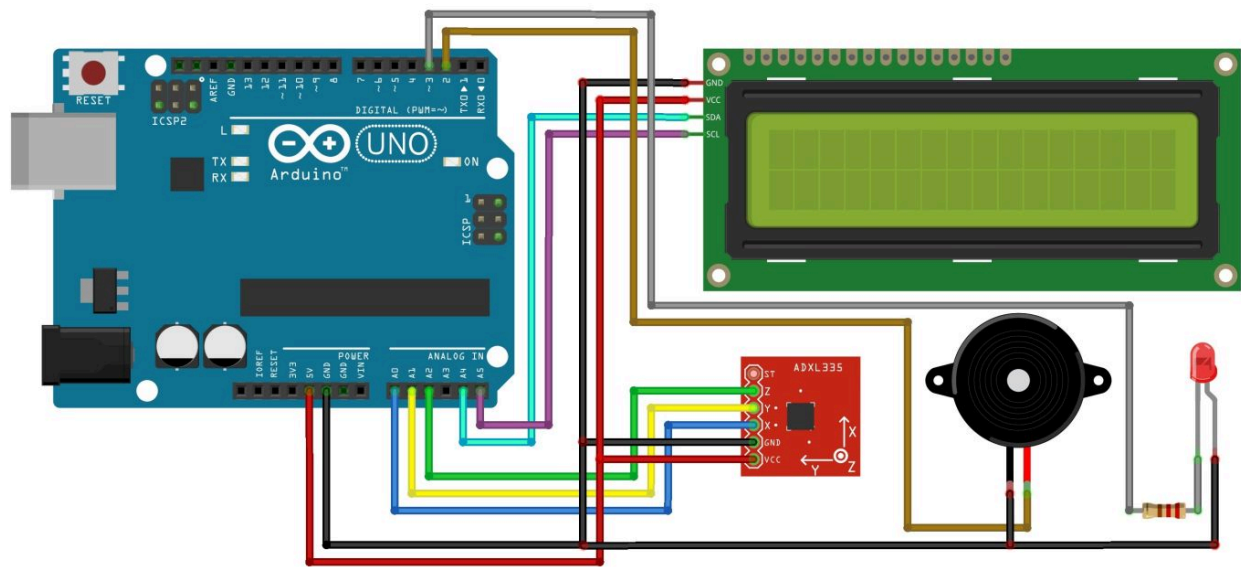


Figure 2. Sample Design

VIII. CODING & DEVELOPMENT

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
#include <math.h>
```

```
// Initialize the LCD with I2C address 0x27, 16 columns, and 2 rows
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
#define buzzer 2 // Pin connected to the buzzer
```

```
#define led 3 // Pin connected to the LED
```

```

// Pins connected to the x, y, and z outputs of the ADXL335 accelerometer
#define xPin A0
#define yPin A1
#define zPin A2

/* Variables */
long xsample = 0;
long ysample = 0;
long zsample = 0;
long start;    // Stores the start time for the buzzer/LED alert

// Variables for Non-Blocking Beeping and Display Restoration
unsigned long beepTimer = 0; // Timer for controlling beep sequence intervals
int displayRestored = 0;    // 0=Needs restoration, 1=Restored/Normal

int buz = 0;    // Flag to indicate buzzer/LED state (on/off)

/* Macros */
#define samples 100    // Number of samples to average during calibration
#define THRESHOLD 6    // Sensitivity
#define buzTime 5000    // Duration for which the buzzer/LED stays on (5 seconds)
#define BEEP_INTERVAL 150 // Time between beeps (milliseconds)
#define BEEP_DURATION 75 // Length of the beep tone (milliseconds)

// Function prototype
String padValue(int value);

// SETUP FUNCTION
void setup() {
    lcd.init();    // Initialize the LCD
    lcd.backlight(); // Turn on the LCD backlight

```

```
Serial.begin(9600); // Initialize serial communication
delay(1000);

// Display initial messages
Serial.println("System Initialized...");
lcd.print("EarthQuake ");
lcd.setCursor(0, 1);
lcd.print("Detector ");
delay(2000);

lcd.clear();
lcd.print("Calibrating.....");
lcd.setCursor(0, 1);
lcd.print("Please wait...");

// Configure buzzer and LED pins as outputs
pinMode(buzzer, OUTPUT);
pinMode(led, OUTPUT);
buz = 0;
digitalWrite(led, LOW);

// Calibration process: Take multiple samples and calculate averages
Serial.println("Calibration Started...");
for (int i = 0; i < samples; i++) {
    xsample += analogRead(xPin); // Read and accumulate X-axis values
    ysample += analogRead(yPin); // Read and accumulate Y-axis values
    zsample += analogRead(zPin); // Read and accumulate Z-axis values
    delay(10);
}

// Calculate the average (baseline) values for X, Y, and Z axes
```

```
xsample /= samples;  
ysample /= samples;  
zsample /= samples;
```

```
// Indicate calibration completion  
delay(1000);  
lcd.clear();  
lcd.print("Calibrated");  
delay(1000);
```

```
Serial.println("Calibration Complete!");  
Serial.print("Baseline X: "); Serial.println(xsample);  
Serial.print("Baseline Y: "); Serial.println(ysample);  
Serial.print("Baseline Z: "); Serial.println(zsample);
```

```
lcd.clear();  
lcd.print("Device Ready");  
delay(1000);
```

```
// Initial display setup (restored display state)  
lcd.clear();  
lcd.print("X-Dif Y-Dif Z-Dif");  
displayRestored = 1; // Display is now in the restored state  
}
```

```
// MAIN LOOP FUNCTION
```

```
void loop() {  
    unsigned long currentMillis = millis(); // Get the current time once per loop
```

```
// --- Reading Sensor Values (RAW) ---  
int value1 = analogRead(xPin);
```



```

int value2 = analogRead(yPin);
int value3 = analogRead(zPin);

// --- Calculating Change (Difference) from Baseline ---
int xValue = abs(xsample - value1);
int yValue = abs(ysample - value2);
int zValue = abs(zsample - value3);

// --- Calculating Magnitude of Vibration (Euclidean distance) ---
long magnitude_sq = (long)xValue * xValue + (long)yValue * yValue + (long)zValue * zValue;
int magnitude = sqrt(magnitude_sq);

// --- CRITICAL DIAGNOSTIC OUTPUT ---
Serial.print("RAW: X="); Serial.print(value1);
Serial.print(" Y="); Serial.print(value2);
Serial.print(" Z="); Serial.print(value3);

Serial.print(" | DIF: X="); Serial.print(xValue);
Serial.print(" Y="); Serial.print(yValue);
Serial.print(" Z="); Serial.print(zValue);
Serial.print(" | Mag: "); Serial.print(magnitude);
Serial.print(" (T="); Serial.print(THRESHOLD); Serial.println(")");

// EARTHQUAKE DETECTION LOGIC
if (magnitude > THRESHOLD) {

    // --- Alert Initialization (Runs only ONCE when magnitude first exceeds threshold) ---
    if (buz == 0) {
        start = currentMillis; // Start timer for alert duration
        beepTimer = currentMillis; // Initialize beep timer
    }
}

```

```

buz = 1;           // Set flag to indicate alert is active
displayRestored = 0; // Display needs restoration after alert ends

Serial.println("--- Earthquake Detected! Alert Activated. ---");

// Display earthquake alert on LCD
lcd.clear();
lcd.print("EARTHQUAKE !!!!");
}

// --- NON-BLOCKING BEEPING LOGIC (Runs continuously during alert) ---
digitalWrite(led, HIGH);

if (currentMillis - beepTimer >= BEEP_INTERVAL) {
    beepTimer = currentMillis; // Reset the timer
    tone(buzzer, 1000, BEEP_DURATION);
}

// Always update the second line with current differences while alert is active
lcd.setCursor(0, 1);
lcd.print("Mag: ");
lcd.print(magnitude);
lcd.print(" T:");
lcd.print(THRESHOLD);

// --- Alert Deactivation (Runs ONCE when buzTime expires) ---
if (currentMillis >= start + buzTime) {
    buz = 0; // Reset alert flag (next loop goes to else block)
    noTone(buzzer); // Stop the tone
    digitalWrite(led, LOW); // Turn off LED
    Serial.println("Alert Deactivated.");
}

```

```

    }

} else {

    // --- Normal Operation (No alert active) ---
    noTone(buzzer); // Ensure buzzer is off if it somehow got stuck
    digitalWrite(led, LOW);

    // --- DISPLAY RESTORATION LOGIC (Runs ONCE when exiting alert state) ---
    if (displayRestored == 0) {
        // Restore normal display headers and clear the screen only once
        lcd.clear();
        lcd.print("X-Dif Y-Dif Z-Dif");
        displayRestored = 1; // Flag as restored
    }

    // Display X, Y, Z difference values on the second line during normal operation
    lcd.setCursor(0, 1);
    lcd.print(padValue(xValue));
    lcd.setCursor(5, 1);
    lcd.print(padValue(yValue));
    lcd.setCursor(10, 1);
    lcd.print(padValue(zValue));
}

delay(200); // Monitoring delay
}

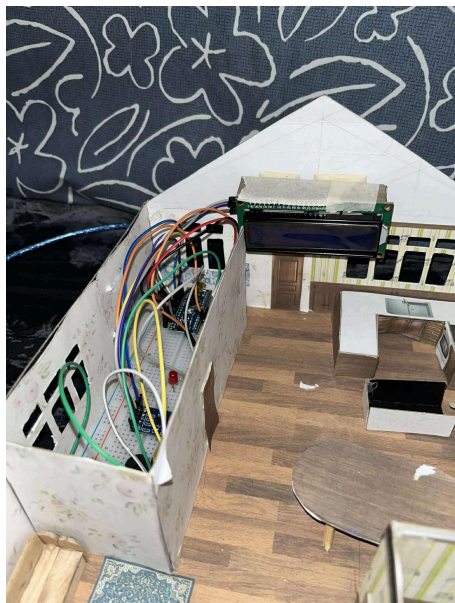
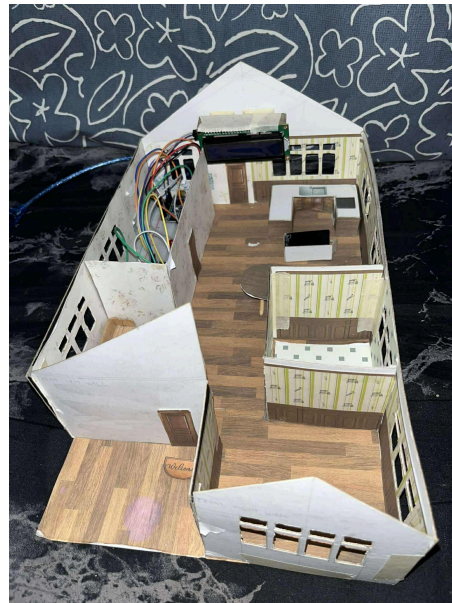
// HELPER FUNCTION
// Function to pad values for LCD display alignment
String padValue(int value) {

```

```
String strValue = String(value);  
// Pad with spaces to ensure proper alignment for 4 characters wide column  
while (strValue.length() < 4) {  
    strValue += " ";  
}  
return strValue.substring(0, 4); // Truncate to ensure exact 4 characters  
}
```

IX. DOCUMENTATION

A. Set-up



B. How to Use/ Operate

Step 1: Placement and Power Up

1. Place the device on a flat, stable surface (e.g., a table or the floor). The device must be completely still before you turn it on.
2. Connect the Arduino to a power source using a USB cable (connected to a laptop or power bank).
3. Do not touch or move the sensor during the first 3 seconds of startup.

Step 2: Automatic Calibration

1. Immediately after receiving power, the LCD screen will light up and display "EarthQuake Detector".
2. The screen will change to "Calibrating..... Please wait...".
3. During this phase, the system is taking 100 rapid readings to determine the "zero" point of gravity. If you bump the table now, the calibration will be wrong, and you will need to press the Reset button on the Arduino to try again.

Step 3: Normal Monitoring Mode

1. Once calibration is complete, the screen will briefly show "Device Ready".
2. The display will then switch to the monitoring screen:

X-Dif Y-Dif Z-Dif

0 1 0

3. How to read this: These numbers show how much the sensor is moving compared to when it started.

- 0, 1, or 2: This is normal "noise." The device is safe.
- High Numbers: The device is tilting or vibrating.

Step 4: Simulating an Earthquake (Testing)

1. Gently shake the device or tap the table firmly to simulate a tremor.
2. Observe the Alert: As long as you are shaking it (and the magnitude is > 5):
 - The Buzzer will beep rapidly.
 - The Red LED will flash.
 - The LCD will display "EARTHQUAKE !!!!" and the magnitude number (e.g., Mag: 5).

Step 5: Stopping the Alarm

1. Stop shaking the device.
2. Immediate Reset: The moment the vibration stops, the alarm will be silenced, the LED will turn off, and the screen will immediately return to the "X-Dif Y-Dif Z-Dif" monitoring view.
3. The system is now instantly ready to detect the next shock.

X. CONCLUSION

The Seismosense project successfully achieved its objective of designing a low-cost and reliable seismic monitoring system using the Arduino Nano and ADXL335 accelerometer. By implementing a vector magnitude algorithm and an auto-calibration feature, the device effectively distinguishes between significant ground tremors and static orientation, addressing the common limitations of simple vibration switches. The integration of immediate visual and auditory alarms ensures that users are promptly alerted to potential earthquake activity, fulfilling the system's goal of real-time detection.

Ultimately, this prototype demonstrates that accessible microcontroller technology can be effectively utilized for personal safety. In a country frequently affected by seismic activity, such

low-cost early warning systems contribute significantly to proactive disaster preparedness and community resilience.

XI. REFERENCES

- Allen, R. M., & Melgar, D. (2019). Earthquake early warning: Advances, scientific challenges, and societal needs. *Annual Review of Earth and Planetary Sciences*. Retrieved December 15, 2025, from <https://www.annualreviews.org/doi/abs/10.1146/annurev-earth-053018-060457>
- Dhanapala, A., Prasanna, R., & Stock, K. (2022). Earthquake early warning systems based on low-cost ground motion sensors: A systematic literature review. *Frontiers in Sensors*. Retrieved December 15, 2025, from <https://www.frontiersin.org/articles/10.3389/fsens.2022.1020202/full>
- Global Facility for Disaster Reduction and Recovery. (2017). Protecting lives and property from disaster risk in the Philippines. *The World Bank*. Retrieved December 15, 2025, from <https://www.gfdrr.org/en/feature-story/protecting-lives-and-property-disaster-risk-philippines>
- Minson, S. E., Brooks, B. A., & Glennie, C. L. (2015). Crowdsourced earthquake early warning. *Science Advances*. Retrieved December 15, 2025, from <https://www.science.org/doi/10.1126/sciadv.1500036>
- Sarker, M. (2020). Vibration Measurement & Analysis Using Arduino Based Accelerometer. *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*. Retrieved December 15, 2025, from http://www.ijareeie.com/upload/2020/july/40_ARDUINO_NC.PDF
- Tanner, A., & Doberstein, B. (2015). Emergency preparedness amongst university students. *International Journal of Disaster Risk Reduction*. Retrieved December 15, 2025, from <https://www.sciencedirect.com/science/article/abs/pii/S221242091500137X>
- Wargantiwar, N. K., Barbade, A. S., Shingade, A. P., & Shire, A. N. (2017). Wireless Earthquake Alarm Design based on MEMS Accelerometer. *International Advanced*

Research Journal in Science, Engineering and Technology. Retrieved December 15, 2025, from

<https://iarjset.com/upload/2017/si/AGNI-PANKH%2017/IARJSET-AGNI-PANKH%2029.pdf>

- Wu, Y. M., Chen, D. Y., Lin, T. L., & Hsieh, C. Y. (2013). A high-density earthquake early warning system in Taiwan based on low-cost sensors. *Seismological Research Letters*. Retrieved December 15, 2025, from <https://pubs.geoscienceworld.org/ssa/srl/article-abstract/84/6/1048/349079>
- Zambrano, A. M., Perez, I., & Palacios, R. (2019). Arduino-Based Earthquake Detector: A DIY Solution for Early Warning. *Journal of Nonlinear Analysis and Optimization*. Retrieved December 15, 2025, from <https://jnao-nu.com/Vol.%2010,%20No.%202:%202019/48.pdf>